

# Traveling

## TravelShare & TravelPath

Application Android - Java


**Étudiants** Sofiane Hammar

Anouar Soufyani


**UE** Programmation mobile

**Année** 2025-2026

**Dépôt Git** `https://github.com/Anouarsoufyani/TravelingApp`

# Table des matières


**1 Introduction** **4**
- 1.1 Contexte du projet . . . 4
- 1.2 Répartition du travail . . . 4
- 1.3 Liste des fonctionnalités . . . 5

**2 Architecture et choix techniques** **7**
- 2.1 Le pattern MVVM . . . 7
- 2.2 Diagramme de composants . . . 7
- 2.3 Rôle de Room vs Firestore . . . 7
- 2.4 Diagramme de séquence — Publication d'une photo . . . 9
- 2.5 Diagramme de séquence — Génération d'un parcours TravelPath . . . 9

**3 Base de données Room v31** **11**
- 3.1 Les 12 entités . . . 11
- 3.2 Diagramme entité-relation . . . 12
- 3.3 Configuration Room . . . 12

**4 Module TravelShare** **14**
- 4.1 Authentification et modes de session . . . 14
- 4.2 Diagramme d'état - Gestion de session . . . 15
- 4.3 Intégration Firebase - FirebaseRepository . . . 15
- 4.4 Cloudinary - Stockage des images . . . 16
- 4.5 Explorateur de photos (ExplorerFragment) . . . 16
  - 4.5.1 Feed et adapter . . . 17
  - 4.5.2 Les 8 filtres . . . 18
- 4.6 Publication (PublishFragment) . . . 19
- 4.7 Consultation d'une photo (PhotoDetailActivity) . . . 20
- 4.8 Profil utilisateur (ProfileFragment) . . . 21
- 4.9 Groupes et Chat (GroupsFragment + GroupChatFragment) . . . 23
  - 4.9.1 Deux types de conversations . . . 23
  - 4.9.2 Chat de groupe . . . 24
- 4.10 Notifications . . . 25
- 4.11 Carte (MapFragment) . . . 27

**5 Module TravelPath** **29**
- 5.1 Le formulaire (TravelPathFragment) . . . 29
- 5.2 L'algorithme de génération . . . 30
  - 5.2.1 Récupération des lieux . . . 31
  - 5.2.2 Les 3 types de plans . . . 31
  - 5.2.3 Lieux obligatoires . . . 32

5.2.4 Géocodage précis . . . . . 32
5.3 Affichage d'un parcours (PlanDetailFragment) . . . . . 33
5.3.1 Carte OSMDroid et tracé OSRM . . . . . 33
5.3.2 Mini-galerie TravelShare par étape . . . . . 33
5.3.3 Météo Open-Meteo . . . . . 33
5.3.4 Sauvegarde et partage . . . . . 33
5.3.5 Export PDF et partage texte . . . . . 34

**6 Passerelle bidirectionnelle** **35**
6.1 TravelShare → TravelPath . . . . . 35
6.2 TravelPath → TravelShare . . . . . 35
6.3 Parcours partagé dans un groupe . . . . . 35

**7 Diagramme de classes UML** **36**

**8 Bilan** **37**
8.1 Ce qu'on a réalisé . . . . . 37
8.2 Récapitulatif technique . . . . . 37

**Annexes** **38**

## Résumé

**Traveling** est une application Android complète autour du voyage. Elle réunit deux expériences complémentaires : partager ses souvenirs avec **TravelShare** et préparer ses sorties avec **TravelPath**.

**TravelShare** est un réseau social dédié aux photos de voyage. On peut publier plusieurs photos d'un coup avec une note vocale et du tagging automatique par IA, les partager en public ou dans un groupe, discuter en temps réel avec d'autres voyageurs, suivre des utilisateurs et recevoir des notifications personnalisées.

**TravelPath** génère des itinéraires de voyage sur mesure. L'utilisateur saisit une ville, ses activités préférées, son budget et sa disponibilité, et l'application lui propose trois parcours différents (économique, équilibré, confort) avec de vrais lieux issus d'OpenStreetMap, un tracé routier calculé par OSRM, la météo du jour et un export PDF. Les parcours peuvent ensuite être partagés directement dans les groupes de discussion.

Techniquement, tout repose sur **MVVM**, une base **Room v31** (12 tables), **Firebase Auth + Firestore** pour le cloud temps réel, **Cloudinary** pour les images, et cinq APIs gratuites : Nominatim, OSRM, Overpass, Open-Meteo et ML Kit.

# 1. Introduction

## 1.1 Contexte du projet

Ce rapport présente **Traveling**, une application Android réalisée en binôme par **Sofiane Hammar** et **Anouar Soufyani**.

L'idée de départ était de créer quelque chose d'utile autour du voyage : partager des souvenirs avec d'autres voyageurs d'un côté, et planifier intelligemment ses sorties de l'autre. Les deux modules se rejoignent naturellement - on peut partir d'une photo vue dans TravelShare pour générer un itinéraire vers cette destination, ou partager un parcours TravelPath dans un groupe de discussion.

## 1.2 Répartition du travail

| **Étudiant** | **Partie** | Fonctionnalités développées |
|---|---|---|
| **Sofiane Hammar** | **TravelShare** | Authentification Firebase (email + anonyme), publication multi-images via Cloudinary, explorateur avec like/partage inline, groupes et conversations directes, système follow/unfollow, notifications, carte OSMDroid, profil complet avec onglets |
| **Anouar Soufyani** | **TravelPath** | Formulaire de génération, vrais lieux via Overpass API, algorithme de parcours (3 types), géocodage Nominatim + Haversine, tracé OSRM, météo Open-Meteo, export PDF, partage de parcours dans les groupes via message de chat |
| **Les deux** | **Architecture** | Base Room v31, ViewModels partagés, intégration Firebase/Firestore, synchronisation Room ↔ Firestore |

## 1.3 Liste des fonctionnalités

**TravelShare - Réseau social de voyages**

**Compte & Session**
- Inscription / connexion via Firebase Auth
- Mode anonyme hybride : Firebase anonymous auth + session locale visiteur
- Session persistante (SharedPreferences)
- Profil public style Instagram
- Photo de profil stockée sur Cloudinary
- Bio éditable, sauvegardée sur Firestore
- Follow / unfollow d'autres utilisateurs
- Abonnés / abonnements cliquables sur le profil

**Publication**
- Multi-images (galerie ou caméra)
- Upload vers Cloudinary (URLs HTTPS publiques)
- Note vocale AAC (enregistrement + lecture)
- Tags IA via ML Kit Image Labeling (seuil 70 %)
- Géocodage automatique via Android Geocoder
- Localisation approximative (±11 km)
- Visibilité PUBLIC ou GROUP (publication dans un groupe)

**Exploration**
- Feed en grands carrés verticaux (ExplorerPostAdapter)
- Like directement depuis le feed
- Partage depuis le feed (groupe, ami, externe)
- 8 filtres : Tous, Nature, Urbain, Culture, Magasin, Autour de moi, Likes, Période
- Recherche texte temps réel + vocale
- Pull-to-refresh + pagination Firestore
- Flux aléatoire par shake ou bouton

**Fiche photo**
- Carousel multi-images (ViewPager2)
- Likes + commentaires temps réel (Firestore)
- Temps de marche OSRM (profil piéton)
- Photos similaires par tags/catégorie
- Signalement de contenu
- Partage externe (WhatsApp, SMS…)
- Partage dans un groupe
- Modifier/supprimer ses posts et commentaires

**Groupes & Chat**
- Groupes publics et privés : accès direct ou demande PENDING → MEMBER
- Conversations directes entre utilisateurs
- Chat temps réel Firestore - 4 types de messages
- Invitation d'amis dans un groupe
- Partage de photos dans le chat
- Partage de parcours TravelPath dans le chat
- Admin : renommer / supprimer le groupe
- Membre : quitter le groupe

**Notifications & Carte**
- Notifications Firestore synchronisées en in-app
- Badge cloche avec compteur de non-lues
- Notification automatique quand un utilisateur suivi publie
- Carte OSMDroid avec pins photos personnalisés

## TravelPath - Générateur d'itinéraires

### Formulaire de génération

- Ville, 5 activités (checkboxes), budget (SeekBar)
- Durée de visite (SeekBar 1–12h)
- Niveau d'effort (Facile / Modéré / Intense)
- Lieux obligatoires avec deux modes : lieux seuls ou lieux + suggestions
- Résultats inline sous le formulaire
- Onglet Sauvegardés (parcours likés)

### Vrais lieux OSM

- Overpass API - lieux nommés uniquement
- Requêtes OSM par activité, avec délai entre appels
- Recherche par nom + ville pour lieux obligatoires
- Fallback Nominatim si Overpass indisponible
- Déduplication des lieux identiques
- Adresse réelle de chaque étape

### Cartographie & Métriques

- Carte OSMDroid + marqueurs par étape
- Tracé routier réel OSRM + durée trajet
- Géocodage Nominatim + validation Haversine
- Météo temps réel Open-Meteo (code WMO)
- Coûts et durées adaptés au type de plan

### Gestion des parcours

- Like = sauvegarde locale + copie Firestore si utilisateur connecté
- Suppression individuelle ou globale
- Régénération avec paramètres préremplis
- Export PDF natif (Canvas Android, format A4)
- Partage texte (Intent.ACTION_SEND)
- Partage dans un groupe via message contenant le `planId`
- Mini-galerie TravelShare par étape
- Mode hors-ligne (Room)

## Passerelle - Intégration bidirectionnelle

- TS → TP : destination transmise automatiquement
- TP → TS : "Voir des photos" par étape
- Navigation unifiée (BottomNavigationView)
- Session partagée entre les deux modules

# 2. Architecture et choix techniques

## 2.1 Le pattern MVVM

Dès le départ, on a choisi d'appliquer rigoureusement le pattern **MVVM** parce que c'est le standard Android recommandé par Google et ça force une vraie séparation entre l'affichage et la logique métier. Concrètement, les Fragments ne font qu'observer des `LiveData` - ils ne savent jamais d'où viennent les données ni comment elles sont récupérées.

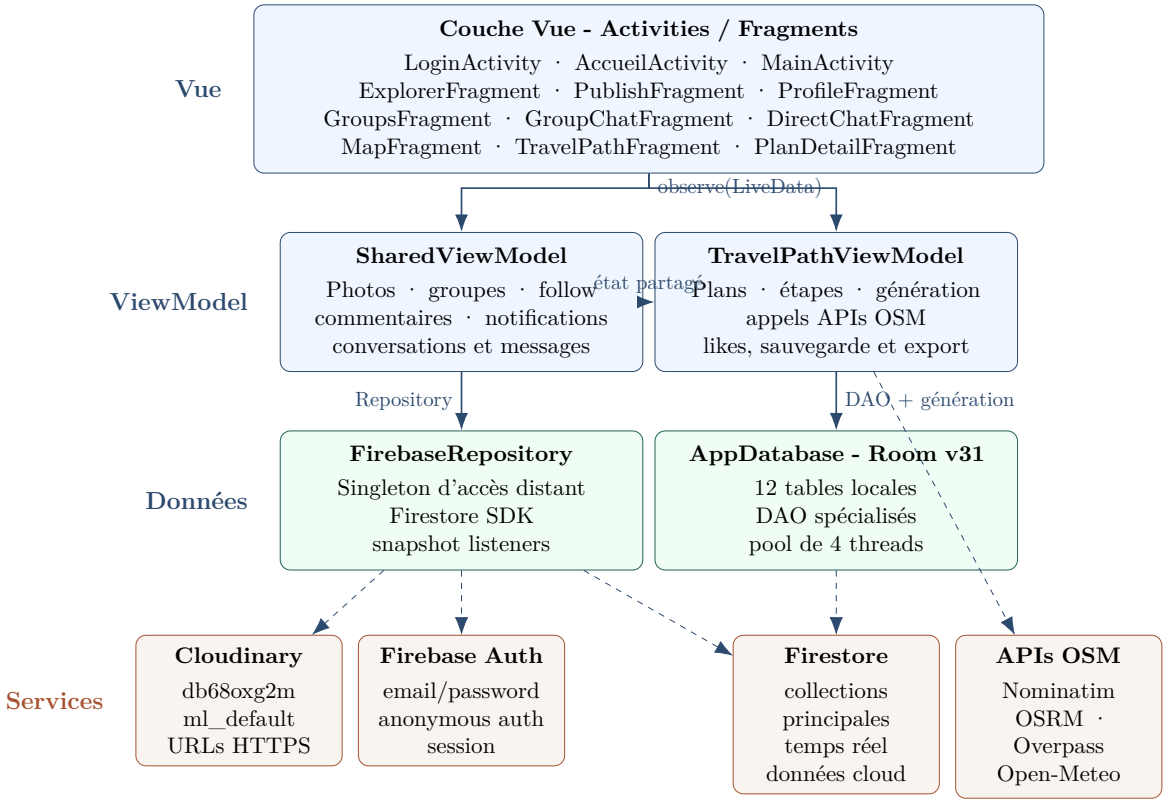



FIGURE 2.1 – Architecture MVVM de l'application Traveling

## 2.2 Diagramme de composants

## 2.3 Rôle de Room vs Firestore

Un point important de notre architecture : Room et Firestore n'ont pas le même rôle selon le module.

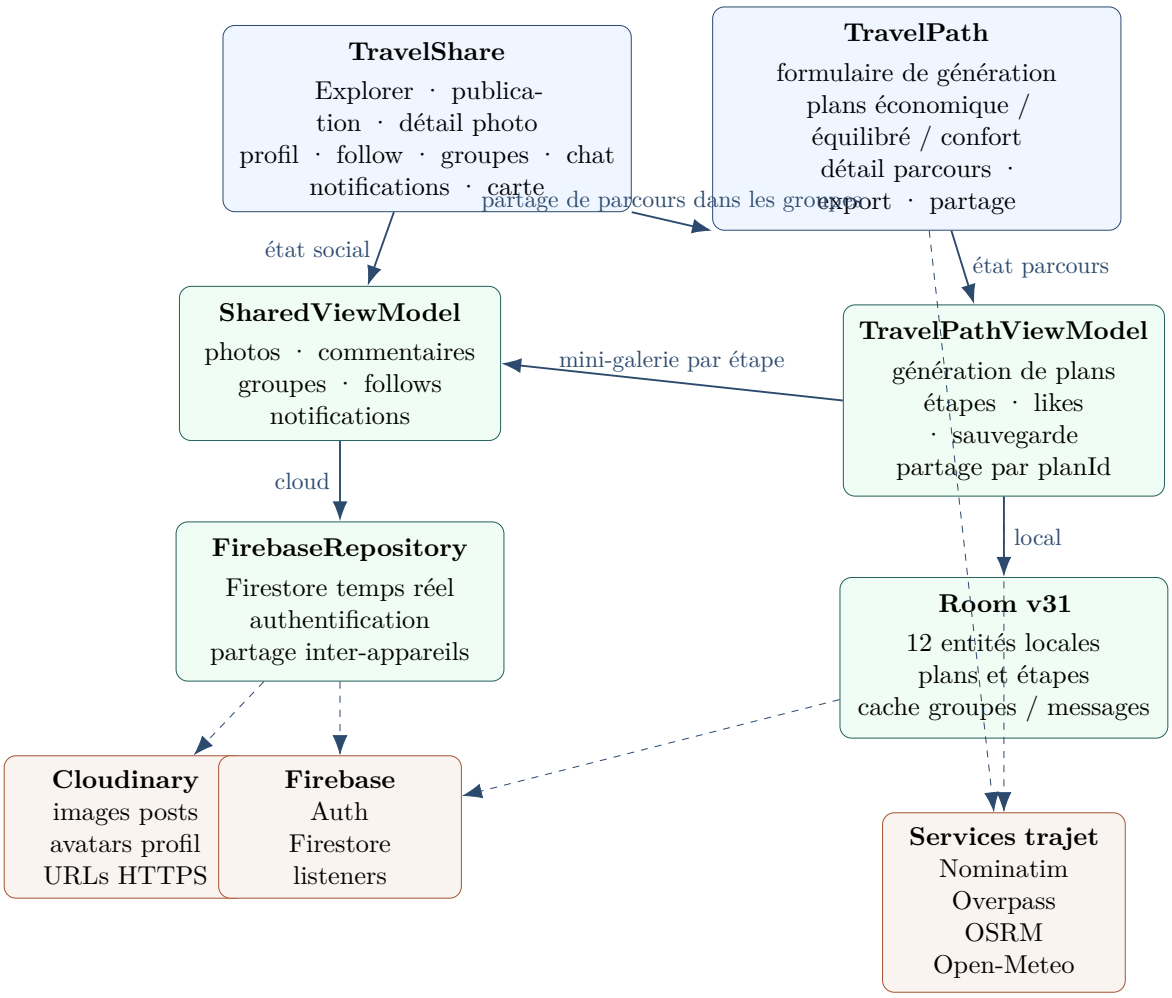



FIGURE 2.2 – Diagramme des composants fonctionnels de Traveling

| Données | Room (local) | Firestore (cloud) |
|---|---|---|
| **Photos** | Vidé au démarrage (`clearPhotos()`) - lecture directe Firestore | Source principale, snapshot listener |
| **Plans TravelPath** | Source principale (génération + affichage) | Copie des métadonnées uniquement si liké |
| **Groupes** | Cache local pour accès rapide | Source de référence, `syncGroupsToRoom()` |
| **Messages chat** | Cache local → `LiveData` met l'UI à jour | Temps réel via `listenToMessages()` |
| **Commentaires** | Cache local | Sync bidirectionnelle |
| **Notifications** | Stockage principal (in-app) | Propagation entre appareils |
| **Utilisateurs** | Cache local du compte | Source principale du profil (bio + avatar Cloudinary) |

On a délibérément choisi de ne plus cacher les photos dans Room - elles ont des URLs Cloudinary qui fonctionnent directement avec Glide sur n'importe quel appareil, donc le cache local n'apporte plus de valeur.

## 2.4 Diagramme de séquence — Publication d'une photo

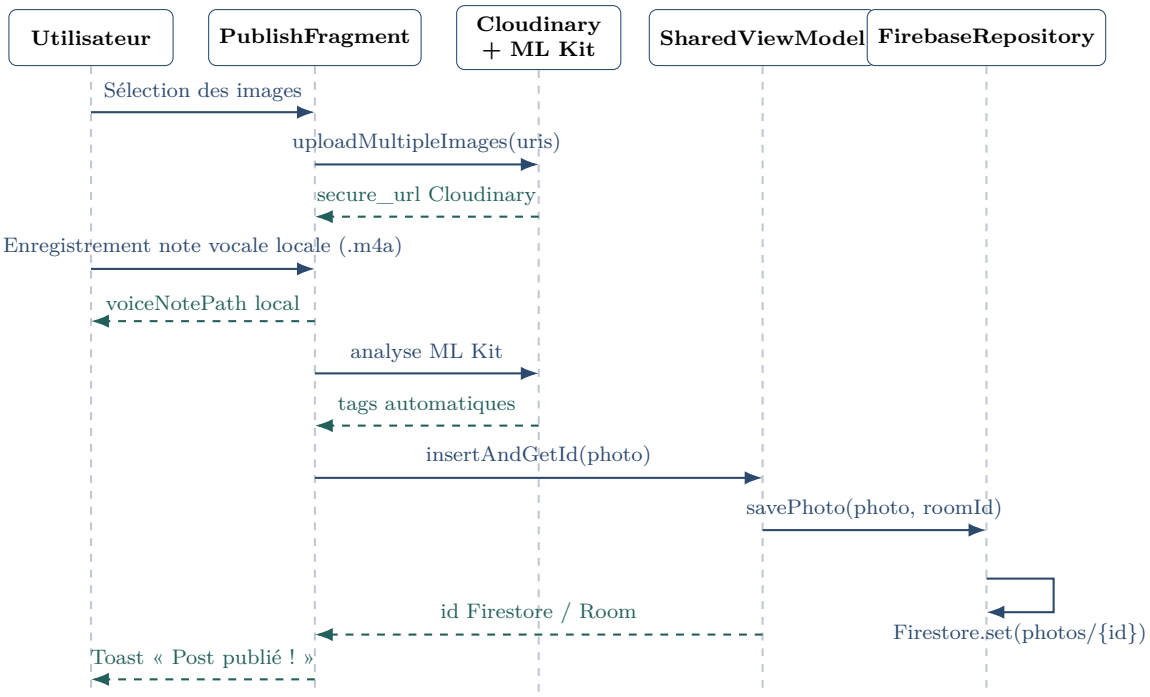



FIGURE 2.3 – Séquence de publication d'une photo TravelShare

## 2.5 Diagramme de séquence — Génération d'un parcours TravelPath

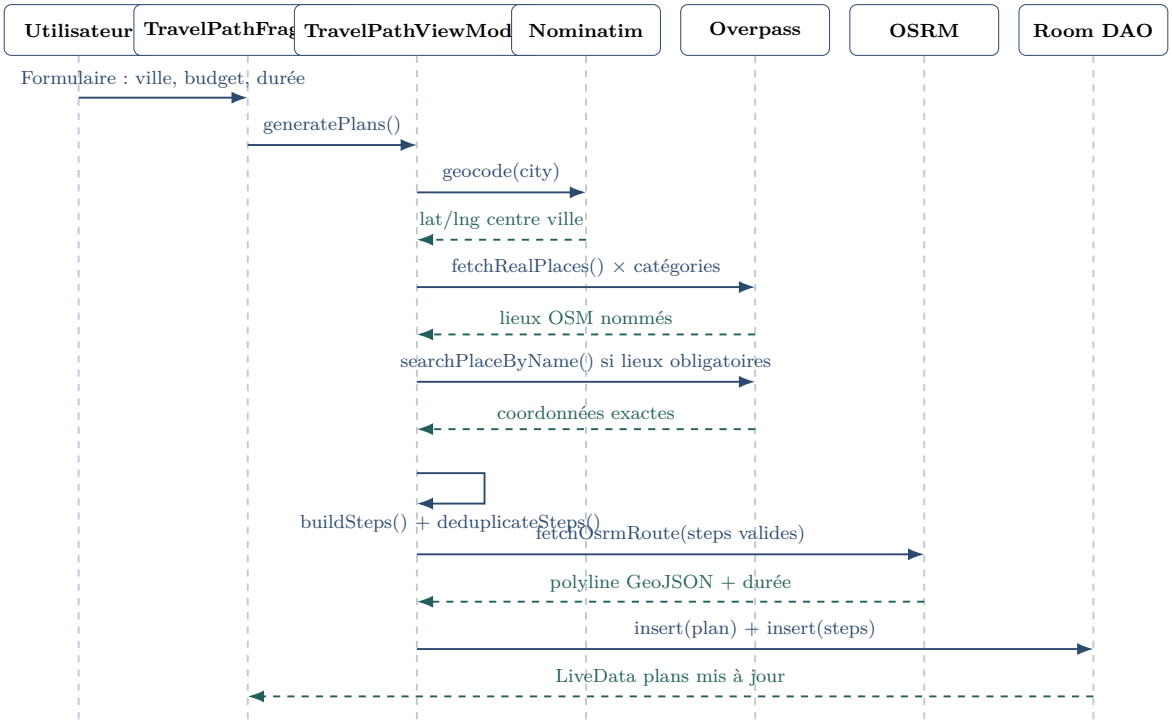



FIGURE 2.4 – Séquence de génération d'un parcours TravelPath

# 3. Base de données Room v31

## 3.1 Les 12 entités

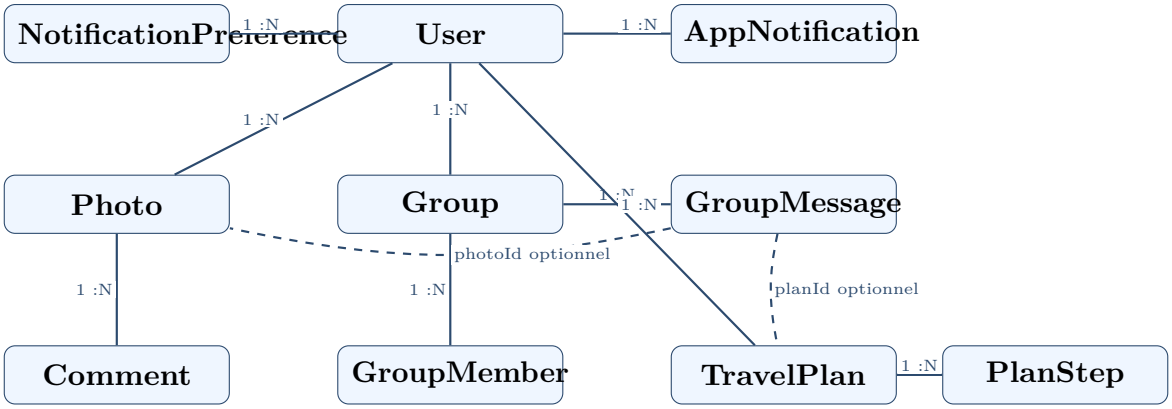
Notre base s'appelle `tp3_database` et est en version 31 avec `fallbackToDestructiveMigration` (on réinitialise la base plutôt que de gérer des migrations complexes, les données cloud étant sur Firestore).

| Entité | Champs et rôle | Module |
|---|---|---|
| `User` | id, login, email, nom, prenom, dateNaissance, telephone, centresInteret, **avatarUri** (URL Cloudinary), **bio**. | Commun |
| `Photo` | id, title, location, author, likes, lat, lng, date, category, tags, visibility (PUBLIC/-GROUP), **imageUri** (URLs Cloudinary séparées par `\|`), groupId, voiceUri. | TS |
| `Comment` | id, photoId, userId, author, text, date. | TS |
| `Group` | **id** (= `stableGroupId(name)`), name, description, creatorId, creatorUsername, visibility. | TS |
| `GroupMember` | id, groupId, userId, userName, status (PENDING / MEMBER). | TS |
| `GroupMessage` | id, groupId, userId, authorName, message, date, **photoId** (photo partagée), **planId** (parcours partagé). | TS |
| `Report` | id, targetId, type, reason, reporterId, date. | TS |
| `NotificationPreference` | id, userId, type, value. Entité conservée dans le schéma Room, mais les alertes voyage ont été remplacées par les notifications automatiques liées aux abonnements. | TS |
| `AppNotification` | id, targetUserId, senderUsername, type, message, groupId, isRead, date. | TS |
| `Planning` | id, userId, title, startDate, endDate. | TP |
| `TravelPlan` | id, userId, city, type (economique / equilibre / confort), activities (CSV), budgetEur, durationHours, effort, liked, saved, requiredPlaces, date. | TP |

| Entité | Champs et rôle | Module |
|---|---|---|
| `PlanStep` | id, planId, stepOrder, name, type, timeSlot, durationMin, costEur, description, lat, lng, address, openingHours. | TP |

TS = TravelShare TP = TravelPath

## 3.2 Diagramme entité-relation




Traits pleins : relations principales Room
Traits pointillés : contenus partagés dans un message

FIGURE 3.1 – Diagramme entité-relation de la base Room v31

## 3.3 Configuration Room

```
@Database(
    entities = {User.class, Planning.class, Photo.class, Comment.class
        ,
                Group.class, GroupMember.class, GroupMessage.class,
                Report.class, NotificationPreference.class,
                AppNotification.class, TravelPlan.class, PlanStep.
                    class},
    version = 31,
    exportSchema = false
)
public abstract class AppDatabase extends RoomDatabase {
    // Pool de 4 threads : toutes les operations DB s'y executent
    public static final ExecutorService databaseWriteExecutor =
            Executors.newFixedThreadPool(4);

    // Singleton thread-safe (double-checked locking)
    public static AppDatabase getInstance(Context context) {
        if (INSTANCE == null) {
            synchronized (AppDatabase.class) {
                if (INSTANCE == null)
                    INSTANCE = Room.databaseBuilder(context,
                                    AppDatabase.class, "tp3_database")
                            .fallbackToDestructiveMigration()
```

```
                                    .build();
            }
        }
        return INSTANCE;
    }
}
```

# 4. Module TravelShare

## 4.1 Authentification et modes de session

L'application démarre sur `LoginActivity` (activité launcher du Manifest). Depuis `AccueilActivity`, l'utilisateur choisit entre "S'inscrire" (→ `MainActivity` avec `OPEN_FRAGMENT = "INSCRI` et "Se connecter" (→ `LoginActivity`).

On supporte trois états de session :

| Mode | Fonctionnement |
|---|---|
| **Connecté** | `signInWithEmailAndPassword(email, password)`. Le username est récupéré depuis Firestore (`collection("users")`), puis depuis Room si Firestore ne répond pas. L'ID de session est `Math.abs(username.toLowerCase().hashCode())` - stable sur tous les appareils. |
| **Anonyme** | `mAuth.signInAnonymously()` tente de créer une identité Firebase temporaire pour les lectures cloud, puis `createAnonymousSession()` crée une session locale visiteur. Si Firebase anonymous échoue, la session locale est quand même créée pour laisser l'utilisateur explorer l'application. `isLoggedIn()` retourne toujours `false` pour un utilisateur anonyme, ce qui bloque les actions d'écriture. |
| **Déjà connecté** | Au démarrage, `LoginActivity` appelle `sessionManager.isLoggedIn()`. Si Firebase contient un utilisateur non anonyme et que la session locale n'est pas marquée visiteur, l'utilisateur est redirigé vers `MainActivity` sans afficher le formulaire. |

Extrait représentatif du mode anonyme : l'application essaie d'abord de créer une session Firebase anonyme, mais le vrai garde-fou applicatif reste la session locale visiteur. Cette séparation permet d'afficher les contenus publics tout en gardant les actions sensibles réservées aux comptes connectés.

Listing 4.1 – LoginActivity.java - entrée en mode anonyme

```
btnAnonymous.setOnClickListener(v -> {
    mAuth.signInAnonymously()
            .addOnSuccessListener(r -> {
                sessionManager.createAnonymousSession();
                goToMain();
            })
            .addOnFailureListener(e -> {
                sessionManager.createAnonymousSession();
```

```
                goToMain();
            });
});
```

## 4.2 Diagramme d'état - Gestion de session

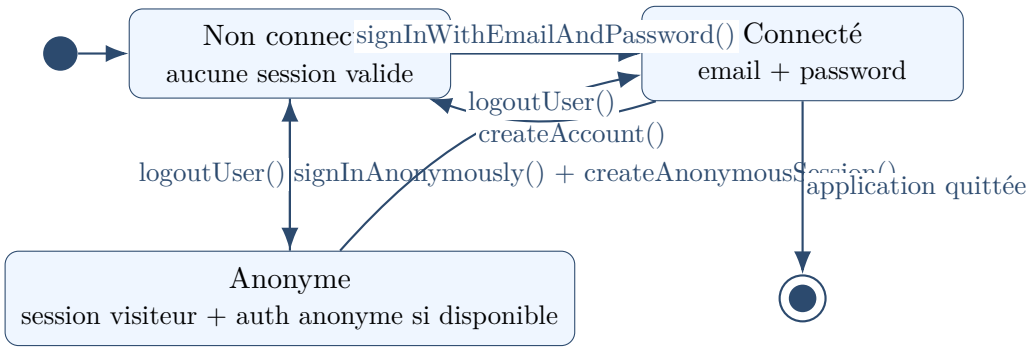



FIGURE 4.1 – Diagramme d'état de la session utilisateur

## 4.3 Intégration Firebase - FirebaseRepository

Toutes les opérations Firestore passent par `FirebaseRepository` (singleton). Les Fragments n'accèdent jamais directement au SDK Firebase - c'est le ViewModel qui fait le lien. L'application utilise plusieurs collections principales :

| **Collection** | Stratégie |
|---|---|
| **photos** | Écriture : `savePhoto(photo, id)` - doc ID = id Room. Lecture : `addSnapshotListener` live + pagination `loadPublicPhotosPage`. Room ne cache plus les photos (vidé au démarrage). |
| **comments** | Écriture Room + Firestore simultanément. `addSnapshotListener` par `photoId` pour les mises à jour temps réel. |
| **users** | `saveUserProfile()` pour bio/avatar. `loadUserAccountByEmail()` à la connexion sur nouvel appareil. Collection `follows` pour le système d'abonnements. |
| **groups** | Sync via `syncGroupsToRoom()` avec `g.id = stableGroupId(name)` pour cohérence avec les `groupId` des notifications. |
| **group_members** | Statut `MEMBER` immédiat pour un groupe public, ou `PENDING` → `MEMBER` pour un groupe privé. |
| **group_messages** | Chat temps réel. Champ `photoId` (photo partagée) et `planId` (parcours partagé). |
| **direct_chats** | Messages directs entre utilisateurs. `listenToMyDirectChats()` récupère toutes les conversations de l'utilisateur. |
| **notifications** | `listenToNotifications()` sur tous les appareils. |
| **plans** | Métadonnées des plans likés : ville, type, budget, durée, effort, date et état liké/sauvegardé. Les étapes restent stockées en Room. |

Exemple d'écriture Firestore pour une publication : on conserve l'identifiant Room comme `roomId`, mais la source de lecture du feed reste la collection `photos`.

Listing 4.2 – FirebaseRepository.java - sauvegarde cloud d'une photo

```
public void savePhoto(Photo photo, long photoId) {
    Map<String, Object> data = new HashMap<>();
    data.put("roomId", photoId);
    data.put("title", photo.getTitle());
    data.put("location", photo.getLocation());
    data.put("author", photo.getAuthor());
    data.put("latitude", photo.getLatitude());
    data.put("longitude", photo.getLongitude());
    data.put("category", photo.getCategory());
    data.put("tags", photo.getTags());
    data.put("visibility", photo.getVisibility());
    data.put("imageUri", photo.getImageUri() != null ? photo.
        getImageUri() : "");
    data.put("createdAtMillis", photo.getCreatedAtMillis() > 0
            ? photo.getCreatedAtMillis() : System.currentTimeMillis())
                ;
    data.put("timestamp", FieldValue.serverTimestamp());

    db.collection("photos").document(String.valueOf(photoId)).set(data
        );
}
```

## 4.4 Cloudinary - Stockage des images

On a choisi Cloudinary plutôt que Firebase Storage parce que Firebase Storage nécessite une carte bancaire même sur le plan Spark, alors que Cloudinary offre un plan gratuit suffisant pour notre projet.

Le fonctionnement est simple : chaque photo sélectionnée est uploadée via `CloudinaryHelper.uploadImage` qui retourne une URL HTTPS permanente (`secure_url`). Cette URL est stockée dans `photo.imageUri`. Pour les publications multi-images, les URLs sont concaténées avec le séparateur `|` (ex : `url1|url2|url3`). La méthode `getImageUriList()` du modèle `Photo` sépare ensuite la chaîne pour afficher les images.

```
// CloudinaryHelper.java
MediaManager.get().upload(imageUri)
    .unsigned(UPLOAD_PRESET)     // "ml_default" - preset non signé
    .callback(new UploadCallback() {
        public void onSuccess(String requestId, Map resultData) {
            String secureUrl = (String) resultData.get("secure_url");
            listener.onSuccess(secureUrl); // URL permanente,
                accessible partout
        }
        public void onError(String requestId, ErrorInfo error) {
            listener.onError(error.getDescription());
        }
    }).dispatch();
```

## 4.5 Explorateur de photos (ExplorerFragment)

### 4.5.1 Feed et adapter

L'explorateur utilise un `RecyclerView` en une colonne de grands carrés (`GridLayoutManager` avec `spanCount = 1`) et `ExplorerPostAdapter`. Chaque item est une carte carrée (`SquareFrameLayout`) avec image `centerCrop`, dégradé sombre en bas, informations superposées (lieu, titre, auteur, tags) et boutons d'action à droite. Le feed permet deux actions sans ouvrir le détail de la photo :

— **Like** : toggle local en `SharedPreferences` + mise à jour Firestore + notification à l'auteur si différent du liker.
— **Partager** : `AlertDialog` proposant groupe, ami (abonnements mutuels), ou partage externe via `Intent.ACTION_SEND`.

Le pattern `observeSource()` garantit qu'une seule source `LiveData` est active à la fois, ce qui évite les conflits entre les différents filtres :

```
private LiveData<List<Photo>> currentSource;

private void observeSource(LiveData<List<Photo>> newSource) {
    if (currentSource != null)
        currentSource.removeObservers(getViewLifecycleOwner());
    currentSource = newSource;
    currentSource.observe(getViewLifecycleOwner(), adapter::setPhotos)
        ;
}
```

L'adapter remplit directement l'overlay et branche les actions inline :

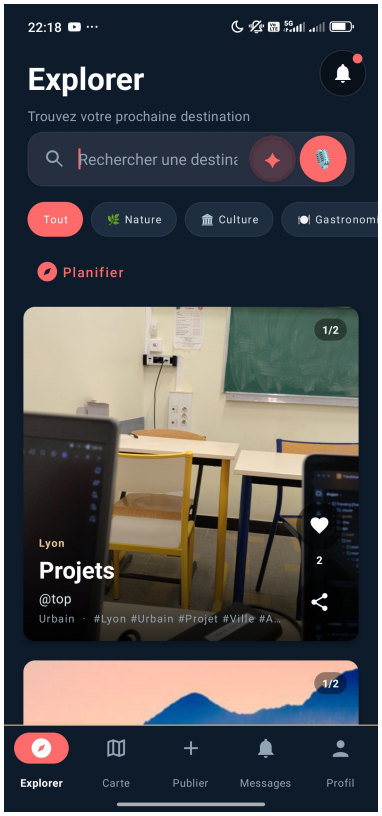
Listing 4.3 – ExplorerPostAdapter.java - carte carrée avec actions inline

```
h.location.setText(photo.getLocation() != null && !photo.getLocation()
    .isEmpty()
        ? photo.getLocation() : "Lieu non renseigné");
h.title.setText(photo.getTitle() != null && !photo.getTitle().isEmpty
    ()
        ? photo.getTitle() : "Sans titre");
h.author.setText("@" + safeAuthor);
h.tags.setText(formatTags(photo));
h.likes.setText(String.valueOf(Math.max(0, photo.getLikes())));
bindLikeState(h, likedIds.contains(photo.getId()));

h.itemView.setOnClickListener(v -> openDetail(v, photo));
h.likeButton.setOnClickListener(v -> {
    if (listener != null) listener.onLike(photo);
});
h.shareButton.setOnClickListener(v -> {
    if (listener != null) listener.onShare(photo);
});
```

### 4.5.2 Les 8 filtres

| Filtre | Implémentation |
|---|---|
| **Tous (défaut)** | Pagination Firestore : `loadPublicPhotosPage(offset, 20)`. `RecyclerView.OnScrollListener` charge la page suivante dès les 4 derniers items visibles. |
| **Catégorie (4 chips)** | Nature, Urbain, Culture, Magasin - 2 rangées fixes de 4 chips via `layout_weight="1"`. Requête `getPhotosByCategoryLive()`. |
| **Période** | `MaterialDatePicker.dateRangePicker()` → `getPhotosByDateRangeLive(start, end)`. |
| **Autour de moi** | `ActivityResultLauncher` pour `ACCESS_FINE_LOCATION`. Delta de 50/111 ≈ 50 km autour de la position GPS. |
| **Mes Likes** | IDs likés en `SharedPreferences` → `getPhotosByIdsLive(ids)` (`WHERE id IN (:ids)`). |
| **Aléatoire** | Bouton dédié + shake (accéléromètre, seuil 12f, cooldown 1500 ms) → `getRandomPublicPhotosLive(10)`. |
| **Recherche texte** | `TextWatcher` → `searchPhotosLive(q)` (filtre Firestore sur titre, auteur, tags). |
| **Recherche vocale** | `RecognizerIntent.ACTION_RECOGNIZE_SPEECH` → résultat injecté dans la barre de recherche → déclenche le `TextWatcher`. |



## 4.6 Publication (PublishFragment)

— **Multi-images** : sélecteur galerie ( `GetMultipleContents` ) ou caméra ( `TakePicture` avec `FileProvider` ). Miniatures en horizontal avec suppression individuelle. Upload séquentiel Cloudinary.
— **Note vocale** : machine d'état à 3 états (IDLE → RECORDING → RECORDED). Même bouton pour démarrer, arrêter et lire. Format MPEG_4/AAC, fichier `.m4a` .
— **Tags IA** : ML Kit Image Labeling lancé dès la sélection d'image. Seuil de confiance 70 % pour éviter les faux positifs. Labels fusionnés avec hashtags manuels.
— **Géocodage** : appel à `android.location.Geocoder` au moment de la publication pour obtenir lat/lng depuis le nom du lieu.
— **Localisation approximative** : `Math.round(lat * 10.0) / 10.0` → précision ±11 km.
— **Choix du groupe** : spinner chargé directement depuis Firestore via `getMyMemberGroups(username)` - fiable sur tout appareil sans dépendre du cache Room.

Le coeur de la publication est volontairement séquentiel : on géocode d'abord le lieu, puis on crée la photo avec les coordonnées, puis on déclenche les notifications destinées aux abonnés.

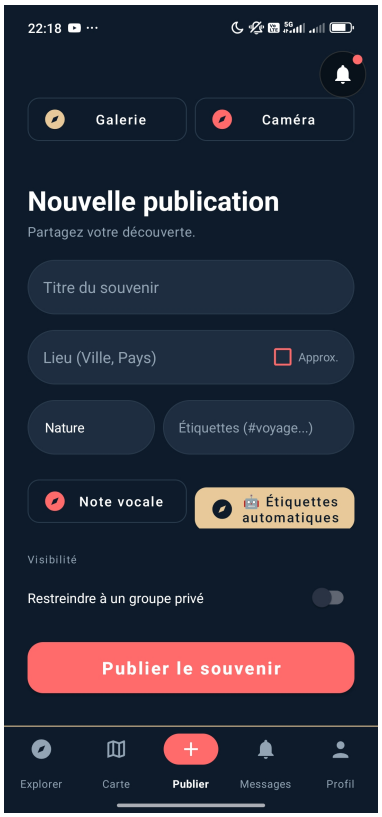
Listing 4.4 – PublishFragment.java - géocodage puis création de publication

```
double[] coords = geocodeLocation(appContext, location);
if (coords == null) {
    btnPublish.setEnabled(true);
    Toast.makeText(getContext(),
            "Lieu introuvable. Précisez la ville ou l'adresse.",
            Toast.LENGTH_LONG).show();
    return;
}

if (approxChecked) {
    coords[0] = Math.round(coords[0] * 10.0) / 10.0;
    coords[1] = Math.round(coords[1] * 10.0) / 10.0;
}

Photo photo = new Photo(title, location, author, 0,
        coords[0], coords[1], date, category, tags, visibility);
if (cloudImageUrl != null) photo.setImageUri(cloudImageUrl);
if (savedVoicePath != null) photo.setVoiceUri(savedVoicePath);
if (finalGroupId >= 0) photo.setGroupId(finalGroupId);

viewModel.insertAndGetId(photo, roomId -> {
    triggerNotificationsForPublish(author, title, location, roomId);
});
```




## 4.7 Consultation d'une photo (PhotoDetailActivity)

La fiche photo affiche les images dans un **carousel ViewPager2** avec un indicateur `TabLayout`. Les URLs Cloudinary sont séparées par `|` et chargées individuellement par Glide.

Actions disponibles :

— **Like / Unlike** : compteur Firestore mis à jour en temps réel via `addSnapshotListener` sur le document.
— **Commenter** : insertion Room + Firestore simultanément. `addSnapshotListener` par `photoId` synchronise les commentaires en temps réel.
— **Signaler** : insertion `Report` en base, bouton désactivé après envoi.
— **Temps de marche** : appel OSRM profil `foot` depuis la position GPS vers les coordonnées de la photo. Affiché en "X min (Y m)".
— **Partager dans un groupe** : `GroupMessage` avec `photoId` renseigné - le chat affiche une carte "Photo partagée" cliquable.
— **Photos similaires** : recherche par le premier tag, fallback par catégorie.
— **Modifier / Supprimer** : visible uniquement pour l'auteur.




## 4.8 Profil utilisateur (ProfileFragment)

Le profil est structuré en deux onglets internes (implémentés avec deux `TextView` stylés en pills, pas un `TabLayout`) :

— **Publications** : grille 3 colonnes (`GridLayoutManager(3)`) via `ProfilePostAdapter`. Clic → `PhotoDetailActivity`.
— **Parcours** : liste linéaire via `ProfilePlanAdapter`. Clic → `PlanDetailFragment`.

Les statistiques du header sont cliquables :

— **Abonnés** : liste des followers depuis Firestore (`collection("follows").whereEqualTo("followed", us`
— **Abonnements** : liste des following.
— **Groupes** : ouvre `GroupsFragment`.

En mode anonyme, le profil affiche "Visiteur / @invité", masque toutes les sections et transforme le bouton de déconnexion en "Se connecter / S'inscrire".

La photo de profil est traitée comme une URL Cloudinary : elle est affichée par Glide, et sa suppression revient à vider l'URL sauvegardée dans le profil cloud. Room peut conserver le champ `avatarUri` dans le modèle `User`, mais la source fonctionnelle du profil visible entre appareils est Firestore (`users.avatarUri`) avec une URL Cloudinary.

Listing 4.5 – FirebaseRepository.java - profil cloud avec avatar Cloudinary

```
1 public void saveUserProfile(String username, String bio, String
    avatarUri) {
2     Map<String, Object> data = new HashMap<>();
3     data.put("username", username);
```

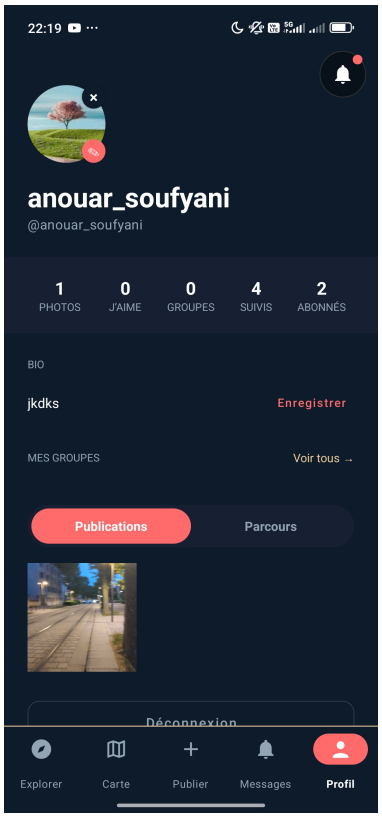
```
    data.put("bio", bio != null ? bio : "");
    data.put("avatarUri", avatarUri != null ? avatarUri : "");
    data.put("timestamp", FieldValue.serverTimestamp());

    db.collection("users")
            .document(username)
            .set(data, com.google.firebase.firestore.SetOptions.merge
               ());
}
```

Listing 4.6 – ProfileFragment.java - affichage et suppression de l'avatar

```
private void setAvatar(String avatarUri) {
    currentAvatarUri = avatarUri != null ? avatarUri.trim() : "";
    if (currentAvatarUri.isEmpty()) {
        Glide.with(this).clear(ivAvatar);
        ivAvatar.setVisibility(View.GONE);
        if (btnDeleteAvatar != null) btnDeleteAvatar.setVisibility(
           View.GONE);
        return;
    }
    ivAvatar.setVisibility(View.VISIBLE);
    Glide.with(this).load(currentAvatarUri).centerCrop().into(ivAvatar
       );
    if (btnDeleteAvatar != null) {
        btnDeleteAvatar.setVisibility(session.isLoggedIn()
                ? View.VISIBLE : View.GONE);
    }
}

private void deleteAvatar() {
    if (!session.isLoggedIn()) return;
    String bio = etBio != null ? etBio.getText().toString().trim() : "
       ";
    setAvatar("");
    FirebaseRepository.getInstance()
            .saveUserProfile(session.getUsername(), bio, "");
}
```




## 4.9 Groupes et Chat (GroupsFragment + GroupChatFragment)

### 4.9.1 Deux types de conversations

`GroupsFragment` gère un flux unifié de conversations dans l'onglet "Mes conversations" via un objet interne `Conversation` qui représente soit un groupe public/privé, soit une conversation directe 1 :1.

**Groupes publics et privés** :

— Création → doc Firestore avec `visibility = PUBLIC` ou `visibility = PRIVATE`, et `id = stableGroupId(name)`.
— Groupe public : clic sur "Rejoindre" → statut `MEMBER` immédiat.
— Groupe privé : clic sur "Demander" → statut `PENDING` + notification `JOIN_REQUEST` envoyée à l'admin, puis validation en `MEMBER`.
— Invitation d'amis directement depuis le chat.
— Admin : renommer (`updateGroupName()` en Room + Firestore), supprimer le groupe.
— Membre : quitter le groupe.

Le choix public/privé est stocké explicitement dans Firestore. Cette information sert ensuite à décider si l'utilisateur rejoint directement le groupe ou s'il envoie une demande en attente.

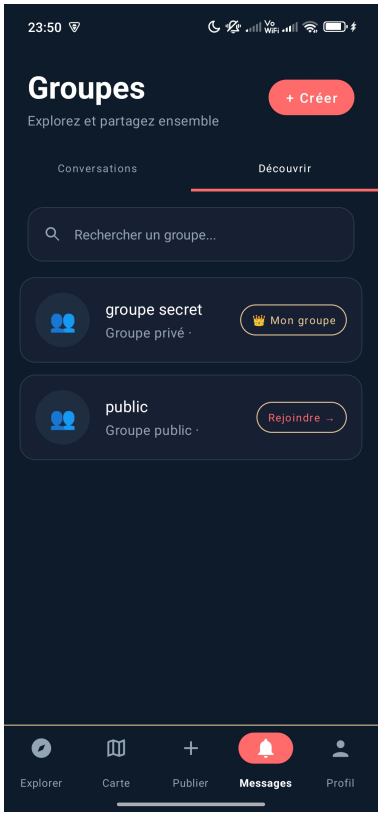
Listing 4.7 – GroupsFragment.java - création et adhésion public/privé

```
g.visibility = rgVisibility.getCheckedRadioButtonId() == rbPublic.
    getId()
        ? Group.VISIBILITY_PUBLIC : Group.VISIBILITY_PRIVATE;

```

```
FirebaseRepository.getInstance().saveGroup(
        name, g.description, session.getUsername(), g.creatorId, g.
            visibility);

boolean isPublic = Group.VISIBILITY_PUBLIC.equals(c.visibility);
if (isPublic) {
    FirebaseRepository.getInstance()
            .saveGroupMember(c.name, session.getUsername(), "MEMBER");
} else {
    FirebaseRepository.getInstance()
            .saveGroupMember(c.name, session.getUsername(), "PENDING")
                ;
    notif.type = "JOIN_REQUEST";
    FirebaseRepository.getInstance().saveNotification(c.
        creatorUsername, notif);
}
```




**Conversations directes** :

— Collection Firestore `direct_chats` avec sous-collection `messages`.
— `listenToMyDirectChats(username)` récupère toutes les conversations de l'utilisateur en temps réel.
— Ouvre `DirectChatFragment` en cliquant sur une conversation.

### 4.9.2 Chat de groupe

Le chat fusionne 4 types de messages dans un seul flux trié par date via `MediatorLiveData` :

| Type | Affichage |
|---|---|
| **TYPE_MESSAGE (0)** | Bulle coral (moi) ou grise (autre). Auteur cliquable → profil. Alignée selon l'émetteur. |
| **TYPE_POST (1)** | Photos publiées directement dans le groupe. Carte avec miniature Cloudinary, titre, lieu, auteur. |
| **TYPE_SHARED (2)** | Photo partagée depuis `PhotoDetailActivity`. Bandeau distinct, miniature chargée via `getPhotoById` Firestore. |
| **TYPE_PLAN (3)** | Parcours TravelPath partagé. Carte "Parcours à X" cliquable → `PlanDetailFragment` si le parcours est présent en Room sur l'appareil. |




## 4.10 Notifications

**Notifications de publication** : après chaque publication, `triggerNotificationsForPublish()` récupère les abonnés de l'auteur et crée une notification Firestore de type `FOLLOW_POST` pour chacun d'eux. Le choix est plus simple pour l'utilisateur : suivre un profil suffit pour être prévenu lorsqu'il publie. Les anciennes alertes par préférence (auteur, lieu, tag, groupe) restent présentes dans le schéma Room, mais elles ne pilotent plus les notifications générées lors d'une publication.

Listing 4.8 – PublishFragment.java - notification automatique aux abonnés

```
1 private void triggerNotificationsForPublish(String author, String
      title,
2                                             String location, long
```

```
                                                    photoId) {
    FirebaseRepository.getInstance().getFollowers(author, followers ->
        {
        if (followers == null || followers.isEmpty()) return;

        for (String follower : followers) {
            if (follower == null || follower.equalsIgnoreCase(author))
                continue;

            AppNotification notif = new AppNotification();
            notif.type = "FOLLOW_POST";
            notif.senderUsername = author;
            notif.message = author + " a publié \"" + title + "\" à "
                + location;
            notif.photoId = (int) photoId;
            FirebaseRepository.getInstance().saveNotification(follower
                , notif);
        }
    });
}
```

**Synchronisation in-app** : les notifications sont écrites dans la collection Firestore `notifications`. `MainActivity` démarre ensuite `listenToNotifications()` pour copier les notifications destinées à l'utilisateur connecté dans la table Room `AppNotification`. L'interface observe cette table avec `LiveData`, triée par date décroissante, ce qui explique l'affichage des plus récentes en premier.

Listing 4.9 – FirebaseRepository.java - écoute et effacement des notifications (extrait simplifié)

```
public ListenerRegistration listenToNotifications(String username,
        AppDatabase localDb, long localUserId) {
    return db.collection("notifications")
            .whereEqualTo("targetUsername", username)
            .addSnapshotListener((query, e) -> {
                if (e != null || query == null) return;
                AppDatabase.databaseWriteExecutor.execute(() -> {
                    for (QueryDocumentSnapshot doc : query) {
                        AppNotification n = new AppNotification();
                        n.targetUserId = localUserId;
                        n.type = doc.getString("type");
                        n.message = doc.getString("message");
                        n.date = doc.getString("date") != null
                                ? doc.getString("date") : "";
                        localDb.appNotificationDao().insert(n);
                    }
                });
            });
}

public void clearNotificationsForUser(String username,
                                      Consumer<Boolean> callback) {
    db.collection("notifications")
            .whereEqualTo("targetUsername", username)
            .limit(450)
            .get()
            .addOnSuccessListener(query -> {
```

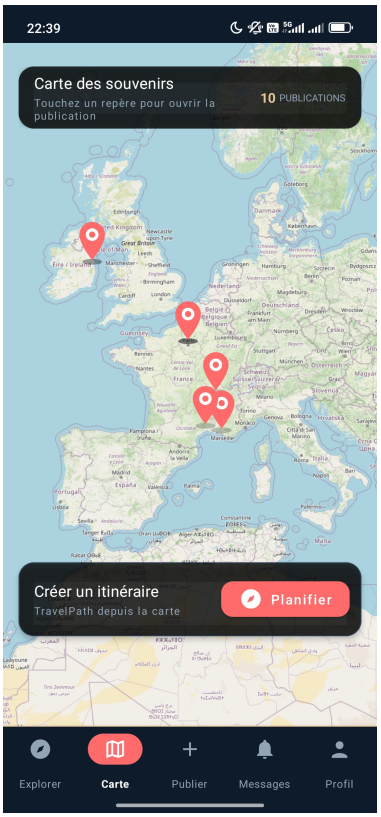
```
                    WriteBatch batch = db.batch();
                    for (QueryDocumentSnapshot doc : query) {
                        batch.delete(doc.getReference());
                    }
                    batch.commit().addOnSuccessListener(unused -> callback
                        .accept(true));
                });
    }
```

Un badge rouge sur la cloche de `MainActivity` compte les non-lues. Le clic sur une notification `FOLLOW_POST` ou `SHARE_POST` ouvre le détail de la photo, tandis qu'une notification `JOIN_REQUEST` ouvre le chat du groupe concerné avec fallback Firestore si le groupe est absent du cache local.




## 4.11 Carte (MapFragment)

OSMDroid avec icône de pin personnalisée (`ic_map_pin_post`). Au clic sur un marqueur, l'application ouvre directement `PhotoDetailActivity` avec les informations de la publication. Le bouton flottant de planification ouvre `TravelPathFragment`. `syncPhotosFromFirestore()` est appelé au chargement pour synchroniser les photos depuis Firestore.


22:39
Carte des souvenirs
Touchez un repère pour ouvrir la publication
10 PUBLICATIONS
Créer un itinéraire
TravelPath depuis la carte
Planifier
Explorer
Carte
Publier
Messages
Profil

# 5. Module TravelPath

## 5.1 Le formulaire (TravelPathFragment)

`TravelPathFragment` a deux onglets gérés par `TabLayout.OnTabSelectedListener` :

— **Formulaire** : saisie des paramètres + section résultats inline en dessous, cachée tant qu'aucun plan n'existe.
— **Sauvegardés** : liste des plans likés via `getSavedPlansForUser(userId)`.

Le bouton "Générer mes parcours" valide que soit des activités sont cochées (mode normal), soit un lieu obligatoire est saisi (mode "lieux seuls"). Un `ProgressBar` et la désactivation du bouton empêchent les doubles soumissions.

Listing 5.1 – TravelPathFragment.java - lancement de génération côté UI

```
btnGenerate.setEnabled(false);
btnGenerate.setText("Génération...");
progressGenerate.setVisibility(View.VISIBLE);

viewModel.generatePlans(userId, city, activities, budget, duration,
        effort, reqPlaces, null, createdCount -> {
    getActivity().runOnUiThread(() -> {
        btnGenerate.setEnabled(true);
        btnGenerate.setText("Générer mes parcours");
        progressGenerate.setVisibility(View.GONE);
    });
});
```




## 5.2 L'algorithme de génération

Toute la génération s'exécute sur le `databaseWriteExecutor` pour ne pas bloquer l'UI. Elle produit jusqu'à 3 plans avec leurs étapes composées de vrais lieux OSM.

Listing 5.2 – TravelPathViewModel.java - génération sur executor Room

```
public void generatePlans(long userId, String city, Set<String>
    activities,
                          int budgetMax, int durationHours, String
                              effort,
                          String requiredPlaces, Set<String>
                              weatherTolerances,
                          GenerationCallback onDone) {
    AppDatabase.databaseWriteExecutor.execute(() -> {
        double[] cityCenter = geocode(city);
        Map<String, String[][]> overpassCache =
                new ConcurrentHashMap<>();

        for (String activity : safeActivities) {
            String[][] places = fetchRealPlaces(
                    city, cityCenter[0], cityCenter[1], activity,
                    sensitiveWeather, 8);
            if (places != null && places.length > 0) {
                overpassCache.put(activity + "_economique",
                        new String[][]{places[0]});
                overpassCache.put(activity + "_equilibre",
                        new String[][]{places[Math.min(2, places.
                            length - 1)]});
                overpassCache.put(activity + "_confort",
                        new String[][]{places[Math.min(4, places.
                            length - 1)]});
```

```
            }
        }
        // Création des plans économique / équilibré / confort.
    });
}
```

### 5.2.1 Récupération des lieux

Pour chaque activité cochée, `fetchRealPlaces()` interroge l'API Overpass avec des requêtes OSM ciblées. Le filtre `["name"]` garantit uniquement des établissements avec un nom officiel :

```
// Exemple pour "Restauration" autour de Paris :
node["amenity"="restaurant"]["name"](48.74,2.20,48.97,2.46);
node["amenity"="cafe"]["name"](48.74,2.20,48.97,2.46);
// Retourne : "Le Procope", "Cafe de Flore", "Brasserie Lipp"...
```

Dans le code, la requête Overpass est construite dynamiquement avec une bounding box autour de la ville, puis envoyée en POST.

Listing 5.3 – TravelPathViewModel.java - requête Overpass

```
String bbox = String.format(Locale.US, "%.4f,%.4f,%.4f,%.4f",
        centerLat - delta, centerLng - delta,
        centerLat + delta, centerLng + delta);

String query = "[out:json][timeout:20];\n"
        + "(\n"
        + "  node[\"" + osmTag + "\"=\"" + osmValue + "\"](" + bbox +
            ");\n"
        + "  way[\"" + osmTag + "\"=\"" + osmValue + "\"](" + bbox + "
            );\n"
        + ");\n"
        + "out center " + count + ";";

URL url = new URL("https://overpass-api.de/api/interpreter");
HttpURLConnection con = (HttpURLConnection) url.openConnection();
con.setRequestMethod("POST");
con.getOutputStream().write(
        ("data=" + URLEncoder.encode(query, "UTF-8")).getBytes("UTF-8"
            ));
```

Si Overpass ne répond pas (timeout de requête 20s, connexion/lecture plus courtes côté `HttpURLConnection`), on bascule automatiquement sur Nominatim ( `fetchNominatimFallback` ). Les lieux résultants sont dédupliqués par nom via `deduplicateSteps()` .

### 5.2.2 Les 3 types de plans

Les lieux Overpass sont organisés dans un cache `overpassCache` par activité. Les trois plans piochent dans ce cache à des index différents : `idxEco = 0` , `idxEquil = min(2, n-1)` , `idxConfort = min(4, n-1)` . Ainsi chaque plan propose un établissement distinct.

| | Économique | Équilibré | Confort |
|---|---|---|---|
| **Budget** | 25 % du max | 60 % | 90 % |
| **Lieux** | index 0 | index 2 | index 4 |
| **Étapes** | base − 1 (Facile) / base / base + 1 (Intense) | idem | idem |

### 5.2.3 Lieux obligatoires

Quand l'utilisateur saisit un nom de lieu précis, `searchPlaceByName()` interroge Overpass avec un filtre regex insensible à la casse sur 16 km autour du centre de la ville. Contrairement à Nominatim qui retourne le centre-ville pour les enseignes, Overpass trouve les **coordonnées GPS exactes** de l'établissement :

```
// node["name"~"Quick",i](bbox);     → position exacte du restaurant
// node["name"~"Eiffel",i](bbox);    → position exacte de la tour
// Fallback Nominatim si Overpass retourne vide
```

La recherche du lieu obligatoire est exécutée avant les lieux suggérés afin que l'étape demandée soit toujours intégrée au parcours généré.

Listing 5.4 – TravelPathViewModel.java - insertion des lieux obligatoires

```
if (requiredPlaces != null && !requiredPlaces.trim().isEmpty()) {
    String[] places = requiredPlaces.split(",");
    for (String pName : places) {
        String p = pName.trim();
        if (p.isEmpty()) continue;

        String[] placeInfo = searchPlaceByName(
                p, city, cityCenter[0], cityCenter[1]);
        if (placeInfo == null || "0".equals(placeInfo[0])) {
            placeInfo = geocodeWithType(p + " " + city,
                    cityCenter[0], cityCenter[1]);
        }
        if (placeInfo == null || "0".equals(placeInfo[0])) continue;

        PlanStep step = new PlanStep();
        step.name = p;
        step.type = osmClassToStepType(placeInfo[2], placeInfo[3]);
        step.description = "Lieu demandé.";
        step.lat = Double.parseDouble(placeInfo[0]);
        step.lng = Double.parseDouble(placeInfo[1]);
        finalSteps.add(step);
    }
}
```

### 5.2.4 Géocodage précis

Nominatim sans contrainte retourne le résultat mondial le plus populaire. On applique deux filtres :

1. **Viewbox** ±0.4° (`bounded=1`) : contraint les résultats à la zone de la ville.
2. **Validation Haversine** : si la distance dépasse 80 km du centre, les coordonnées sont rejetées (retour 0, 0 → pas de marqueur).

## 5.3 Affichage d'un parcours (PlanDetailFragment)

### 5.3.1 Carte OSMDroid et tracé OSRM

Pour chaque plan, la carte OSMDroid affiche un marqueur par étape valide. Si au moins 2 étapes ont des coordonnées non nulles, `fetchOsrmRoute()` calcule le tracé routier réel en arrière-plan :

```
// URL OSRM : /route/v1/driving/lng1,lat1;lng2,lat2?geometries=geojson
// Réponse : routes[0].geometry.coordinates (tableaux [lng, lat])
// → conversion en GeoPoint(lat, lng) pour OSMDroid
// → Polyline avec stroke navy (#1B3A5C), épaisseur 9dp, cap ROUND
// La durée routes[0].duration est affichée dans le bloc "TRAJET"
```

### 5.3.2 Mini-galerie TravelShare par étape

Chaque étape contient un `RecyclerView` horizontal qui charge des photos TravelShare correspondant au type de l'étape. Si aucune photo n'est trouvée en base, le lien "Voir des photos" ouvre `ExplorerFragment` avec le nom de l'étape en requête - c'est la passerelle TP → TS.

### 5.3.3 Météo Open-Meteo

1. Géocodage de `plan.city` via Nominatim → lat/lng.
2. Appel `https://api.open-meteo.com/v1/forecast?current_weather=true`.
3. Parse du bloc JSON `current_weather` → température (°C) + code WMO.
4. Affichage : icône + description + conseil selon les conditions.

| Code WMO | Icône | Conseil |
|---|---|---|
| 0 | soleil | Ciel dégagé - Parfait pour visiter |
| 1–3 | nuage | Partiellement nuageux - Bonne journée |
| 45–49 | - | Brouillard - Visibilité réduite |
| 51–67 | - | Pluie - Prévoyez un imperméable |
| 71–77 | - | Neige - Habillez-vous chaudement |
| 80–82 | - | Averses - Parapluie conseillé |
| 95+ | - | Orage - Préférez les activités intérieures |

### 5.3.4 Sauvegarde et partage

Quand un plan est liké, `saveLikeAndSave()` met à jour Room puis appelle `savePlan(username, plan)` si l'utilisateur est connecté. La copie Firestore contient les métadonnées du plan (ville, type, budget, durée, effort, date, état liké/sauvegardé). Les étapes détaillées restent stockées localement dans Room, ce qui garde l'affichage principal rapide et hors-ligne.

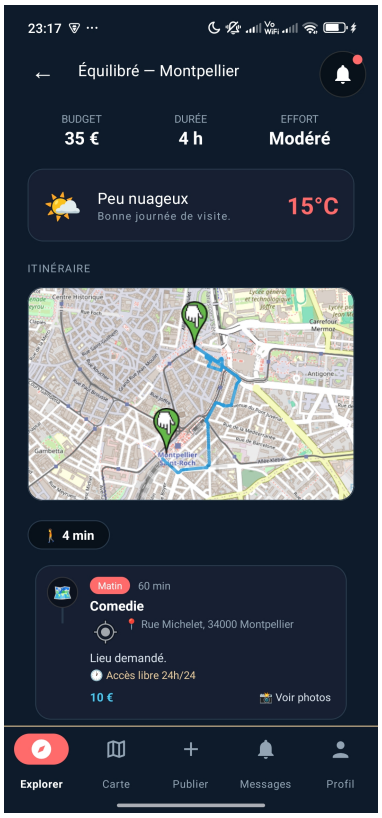
Listing 5.5 – TravelPathViewModel.java - sauvegarde d'un parcours liké

```
public void saveLikeAndSave(TravelPlan plan) {
    AppDatabase.databaseWriteExecutor.execute(() -> {
        SessionManager session = new SessionManager(getApplication());
        if (!session.isLoggedIn()) return;

        planDao.updatePlan(plan);
```

```
        if (plan.liked) {
            String username = session.getUsername();
            if (username != null && !username.isEmpty()) {
                FirebaseRepository.getInstance().savePlan(username,
                    plan);
            }
        }
    });
}
```

Le partage dans un groupe envoie un `GroupMessage` contenant le `planId`. Le chat affiche alors une carte "Parcours à X". L'ouverture du détail repose sur le plan local correspondant : c'est cohérent avec le choix architectural où les étapes TravelPath restent la source Room.




### 5.3.5 Export PDF et partage texte

Le bouton "Exporter PDF" génère un document `PdfDocument` Android natif (Canvas A4, 595×842 pt) avec titre, étapes, horaires et coûts. Sauvegardé dans `Documents/TravelPath/` et ouvert via `FileProvider + Intent.ACTION_VIEW`.

Le bouton "Partager" propose trois destinations : un groupe (message de chat avec `planId`), un ami (abonné mutuel via `getFriends()`), ou un partage texte externe (`Intent.ACTION_SEND`).

# 6. Passerelle bidirectionnelle

## 6.1 TravelShare → TravelPath

Depuis l'explorateur, un bouton "Planifier un voyage" transmet la destination saisie dans la barre de recherche au formulaire TravelPath via `ARG_CITY`. Une bannière verte confirme l'import de la destination.

## 6.2 TravelPath → TravelShare

Dans le détail d'un parcours, chaque étape affiche une mini-galerie de photos TravelShare de même catégorie. Le lien "Voir des photos" ouvre l'explorateur avec le nom de l'étape pré-rempli.

## 6.3 Parcours partagé dans un groupe

C'est la passerelle la plus simple entre les deux modules côté chat : un utilisateur partage son parcours dans un groupe, les membres reçoivent une carte "Parcours à X" contenant le `planId`. L'ouverture complète du détail reste liée à la présence du plan et de ses étapes dans la base Room locale.

# 7. Diagramme de classes UML

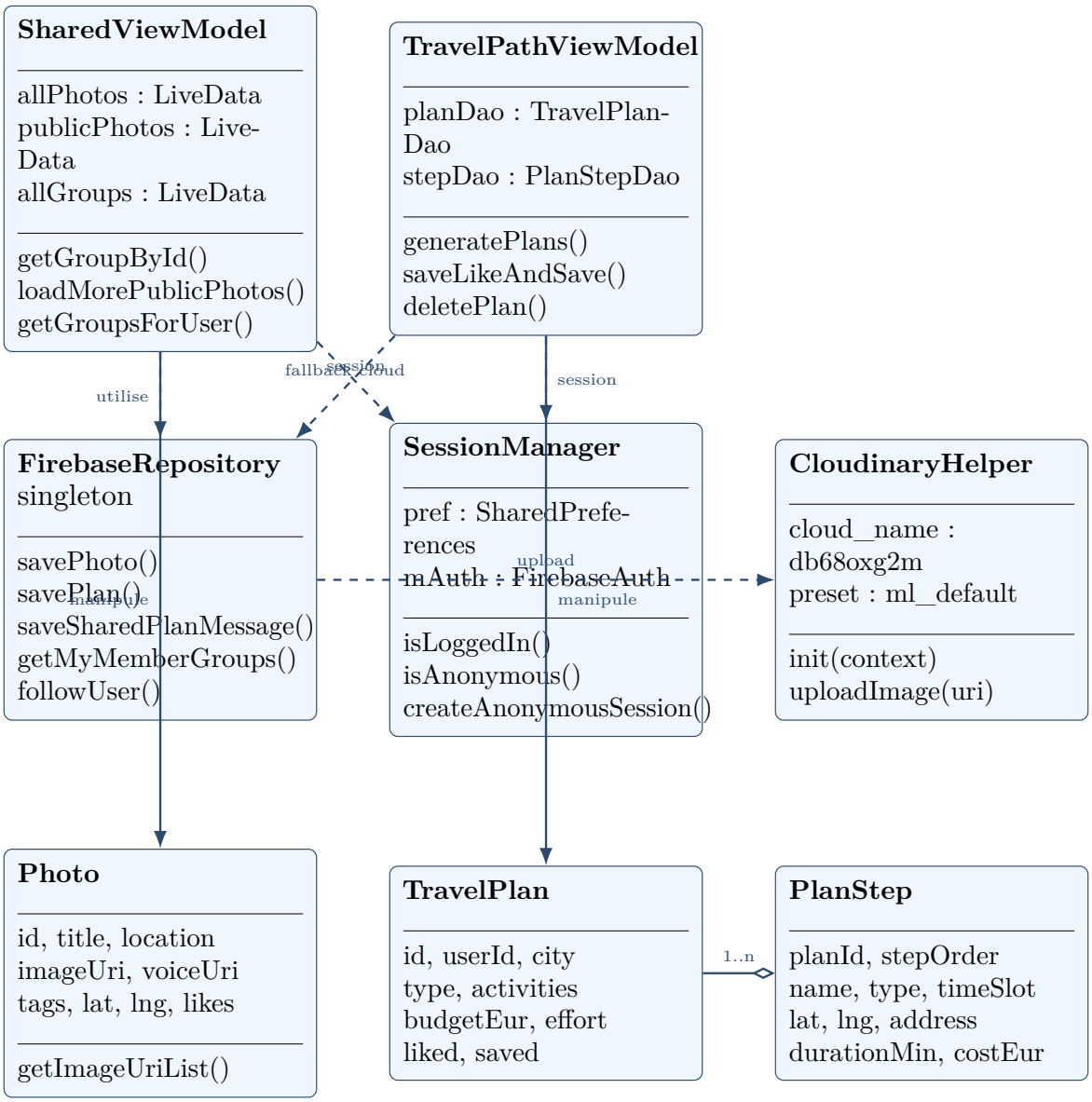



FIGURE 7.1 – Diagramme de classes simplifié de l'application

# 8. Bilan

## 8.1 Ce qu'on a réalisé

En partant de zéro, on a construit une application Android complète en Java avec deux modules qui fonctionnent ensemble. Techniquement, ce qui nous a demandé le plus de travail c'est la synchronisation entre Room et Firestore : trouver le bon équilibre entre lecture locale rapide et données cloud à jour n'est pas trivial. Par exemple, les photos sont maintenant lues directement depuis Firestore (plus de cache Room) parce que les URLs Cloudinary sont universelles, alors que les plans TravelPath restent en Room pour l'accès hors-ligne.

Un autre point important a été le partage de parcours entre utilisateurs : le chat transporte une carte de parcours via un `planId`, tandis que les étapes complètes restent en Room. Ce choix garde TravelPath robuste en hors-ligne, tout en permettant de signaler un parcours dans une conversation.

## 8.2 Récapitulatif technique

| **Aspect** | Détail |
|---|---|
| **Langage** | Java 8 (Android) |
| **Architecture** | MVVM - Fragments, ViewModels, LiveData, Repository |
| **Base locale** | Room v31 - 12 entités - pool 4 threads |
| **Cloud** | Firebase Auth + Firestore (collections principales, Spark gratuit) |
| **Images** | Cloudinary (cloud_name : db68oxg2m, preset : ml_default) |
| **Cartographie** | OSMDroid 6.1.18 |
| **APIs externes** | Nominatim · OSRM · Overpass · Open-Meteo · ML Kit |
| **Chargement images** | Glide 4.16.x |
| **SDK cible** | API 34 (Android 14) - minimum API 24 (Android 7) |

# Annexes

## A - Dépôt Git

— **URL** : `https://github.com/Anouarsoufyani/TravelingApp`
— **Branche principale** : `main`

## B - APIs et services utilisés

| Service | Rôle | Coût |
|---|---|---|
| Firebase Auth | Auth email/password + mode anonyme | Gratuit (Spark) |
| Firebase Firestore | Synchronisation cloud temps réel | Gratuit (Spark) |
| Cloudinary | Stockage images HTTPS permanent | Gratuit |
| Nominatim (OSM) | Géocodage adresses → GPS | Gratuit |
| OSRM | Calcul itinéraires routiers réels | Gratuit |
| Open-Meteo | Météo temps réel (code WMO) | Gratuit |
| Overpass API (OSM) | Recherche lieux nommés OSM | Gratuit |
| ML Kit (Google) | Tagging IA on-device | Gratuit |
| OSMDroid | Cartographie Android | Open-source |
| Glide | Chargement + cache d'images | Open-source |

## C - Permissions Android

— `android.permission.INTERNET`
— `android.permission.ACCESS_NETWORK_STATE`
— `android.permission.ACCESS_FINE_LOCATION`
— `android.permission.ACCESS_COARSE_LOCATION`
— `android.permission.RECORD_AUDIO`
— `android.permission.POST_NOTIFICATIONS`

## D - Environnement de développement

| **Outil** | Version |
|---|---|
| **Android Studio** | Hedgehog 2023.1+ |
| **SDK Android** | API 34 (compile/target), API 24 (minimum) |
| **Java** | Java 8 |
| **Room** | 2.6.x (version base de données : 31) |
| **OSMDroid** | 6.1.18 |
| **Glide** | 4.16.x |